



Architecture Styles & Trade-Offs

From **Best** Architecture to **Right** Architecture

Software Architecture

Misconceptions:

- “Microservices are modern → we must use them”
- “Clean Architecture everywhere”
- Architecture chosen before understanding the problem

Reality:

- Architectures exist to manage **constraints**, not to look elegant
- Every architecture optimizes *something* and sacrifices *something else*

Monolithic Architecture

Nature:

- Single deployable unit
- Shared codebase, shared database

Strengths:

- Simple to understand
- Easy to test end-to-end
- Fast development early
- Minimal operational overhead

Monolithic Architecture ...

Costs:

- Tight coupling grows over time
- Vertical Scaling
- Deployment risk increases

Suitable for:

- Early-stage products (or MVP)
- Small teams
- Low operational maturity
- Need fast feedback

Note: A well-structured monolith is not a failure.

Layered Architecture

Structure:

- Presentation/ Controller
- Application / Service
- Domain / Core
- Infrastructure / Data

What It Optimizes:

- Separation of concerns
- Team onboarding
- Testability (to a degree)
- Establishes a convention

Onion Architecture

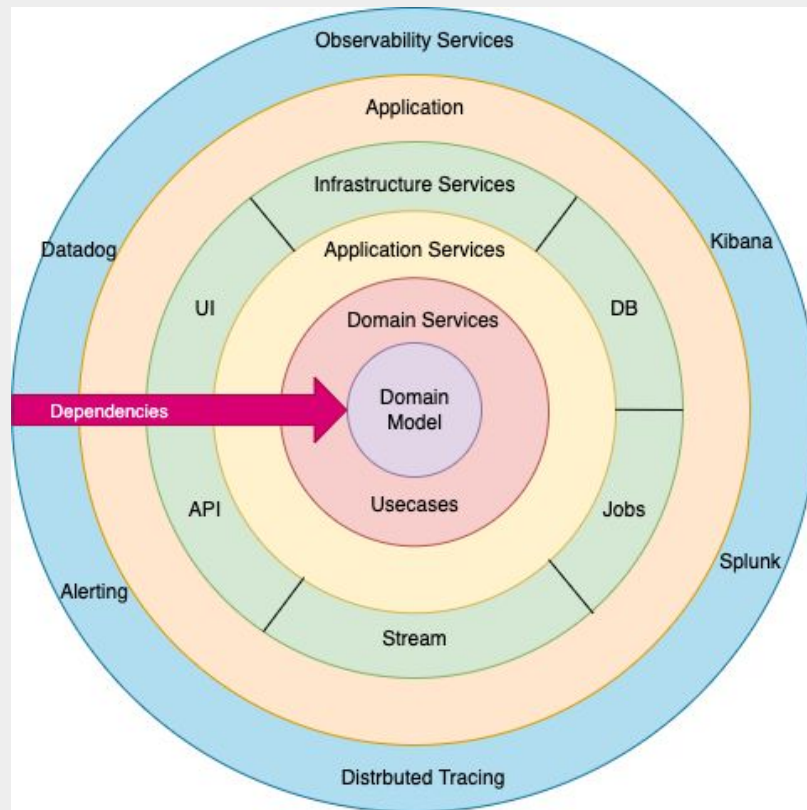
Core Idea:

- Domain at the center
- Dependencies point inward

Benefits:

- Domain independence
- Infrastructure replaceable
- Improved testability

Onion Architecture



Hexagonal Architecture (Ports & Adapters)

Key Concept:

- Application core is isolated from external systems

Ports:

- Interfaces the core defines

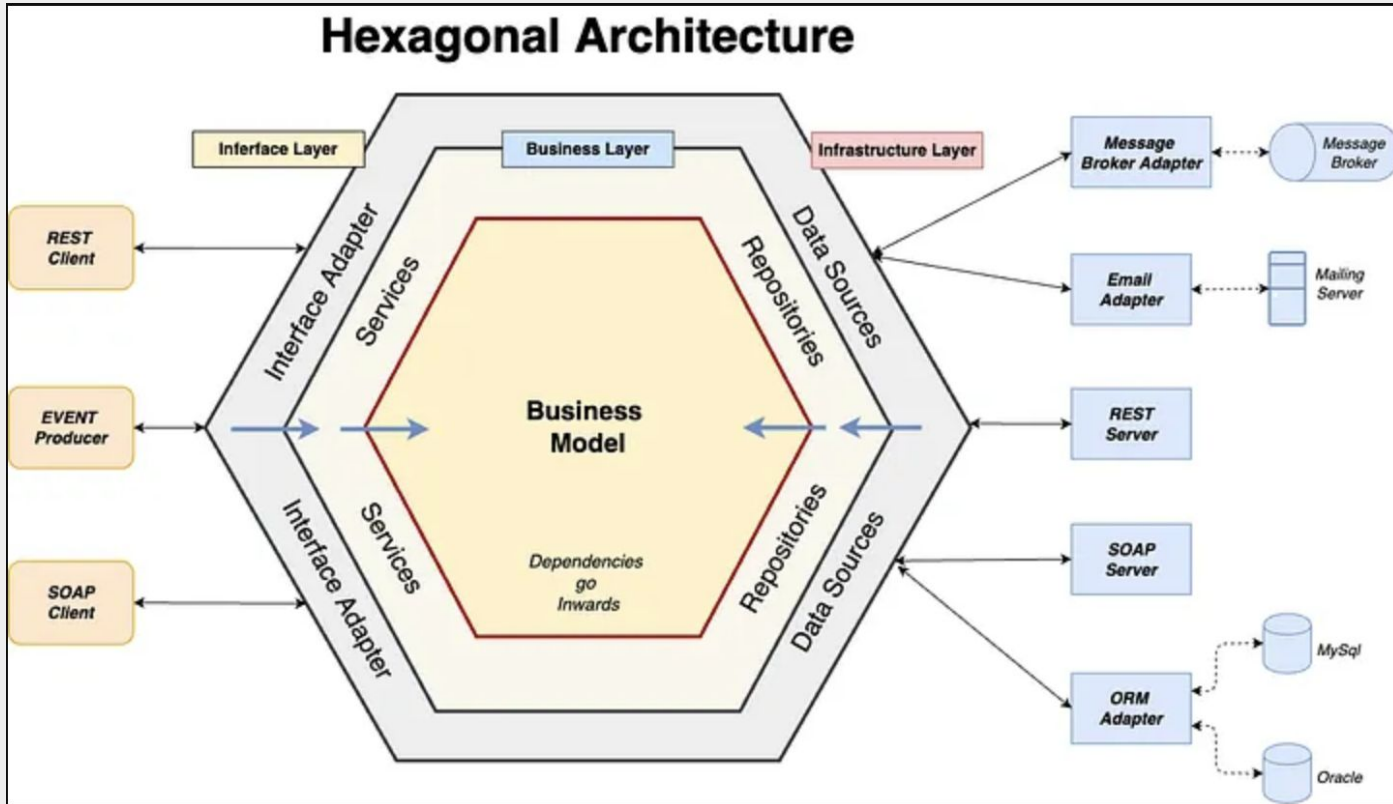
Adapters:

- Implementations (DB, REST, MQ)

Best for:

systems with many diverse external integrations (APIs, message queues, multiple DB types) where flexibility at the edges is the priority.

Hexagonal Architecture... ..



Service Oriented Architecture (SOA)

Definition:

- Having different Services
- Each service solves a specific part of business not a technical domain like micro-service
- Communicate over the network call
- Shared database
- Example: Account Service, Loan Service etc.

Benefits:

- Centralized security & policy enforcement
- Works well in regulated environments
- Centralized control and governance

Microservices Architecture

Definition:

- Independently deployable services
- Aligned to business capabilities
- Bounded responsibilities for each service(micro)

Benefits:

- Independent scaling
- Fault isolation (in theory)
- Easier to test

Microservices Reality Check

They require:

- Strong DevOps
- Mature monitoring
- Domain-driven boundaries (**will cover it in later session**)

When NOT to Use Microservices:

- Small team
- No DevOps automation
- Unclear domain boundaries
- Low traffic
- Limited observability
- Lack of standard practices

Event-Driven Architecture

Core Idea:

- Systems react to events
- Service level loose coupling
- Own data with independency
- No network call but message bus
- Eventual consistency

Cons:

- Debugging high complexity
- Event versioning
- Eventual consistency

Architecture as a Series of Trade-Offs

There is no “best” architecture.

There is:

- Best for requirements
- Best for available resources (human, server, budget)
- Best for failure management etc

Note: Architecture should evolve, not be married upfront.



Thank you!